

Angular + Spring Boot Full Workflow

1 Overall Architecture

Frontend (Angular) - Component → Service → HttpClient → Backend API

Backend (Spring Boot) - Controller → Service → Repository → Database

```
graph TD
    User["User (Browser)"] --> AngularComp["Angular Component"]
    AngularComp --> AngularService["Angular Service"]
    AngularService --> HttpClient["HttpClient (HTTP Request)"]
    HttpClient --> SpringBootController["Spring Boot Controller"]
    SpringBootController --> ServiceLayer["Service Layer"]
    ServiceLayer --> RepositoryLayer["Repository Layer"]
    RepositoryLayer --> Database["Database"]
    Database --> Response["Response (JSON)"]
    Response --> AngularService2["Angular Service"]
    AngularService2 --> AngularComp2["Angular Component"]
    AngularComp2 --> UIUpdates["UI Updates"]
```

2 Example Use Case

Goal: Display a list of users **Backend API:** GET http://localhost:8084/users

3 Backend Implementation

3.1 Entity

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    // getters and setters
}
```

3.2 Repository

```
@Repository
public interface UserRepository extends JpaRepository<User,Long>{}
```

3.3 Service Layer

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
    public List<User> getUsers(){
        return userRepository.findAll();
    }
}
```

3.4 Controller

```
@RestController
@RequestMapping("/users")
public class UserController {
    @Autowired
    private UserService userService;

    @GetMapping
    public List<User> getUsers(){
        return userService.getUsers();
    }
}
```

4 Backend Execution Flow

```
GET /users
  ↓
UserController
  ↓
UserService
  ↓
UserRepository
  ↓
Database
  ↓
List<User>
  ↓
JSON Response
```

Example Response:

```
[
  { "id":1, "name":"Arun"},
  { "id":2, "name":"Rahul"}
]
```

5 Frontend Implementation

5.1 Angular Service

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private apiUrl = "http://localhost:8084/users";

  constructor(private http: HttpClient) {}

  getUsers(){
    return this.http.get(this.apiUrl);
  }
}
```

5.2 Angular Component

```
export class UserListComponent implements OnInit {  
  users:any;  
  constructor(private userService:UserService){}  
  
  ngOnInit(){  
    this.userService.getUsers().subscribe(data=>{  
      this.users = data;  
    });  
  }  
}
```

5.3 HTML View

```
<ul>  
  <li *ngFor="let user of users">  
    {{user.name}}  
  </li>  
</ul>
```

6 Angular Execution Flow

```
User opens page  
↓  
Angular Router loads component  
↓  
constructor()  
↓  
ngOnInit()  
↓  
Component calls Service  
↓  
Service calls HttpClient  
↓  
HTTP Request sent to backend
```

7 Full End-to-End Flow

```
User opens page
↓
Angular Component loads
↓
ngOnInit()
↓
UserService.getUsers()
↓
HttpClient GET request
↓
Spring Boot Controller
↓
Service
↓
Repository
↓
Database
↓
JSON Response
↓
Angular Service receives data
↓
Component subscribe()
↓
UI updates
```

8 Angular Lifecycle Hooks Example

```
@Component({selector:'app-lifecycle-demo', template:`<p>{{userName}}</p>`})
export class LifecycleDemoComponent implements OnInit, OnChanges, DoCheck,
AfterViewInit, OnDestroy {
  @Input() userName!: string;

  constructor(){ console.log("Constructor called"); }
  ngOnChanges(){ console.log("ngOnChanges called"); }
  ngOnInit(){ console.log("ngOnInit called"); }
  ngDoCheck(){ console.log("ngDoCheck called"); }
  ngAfterViewInit(){ console.log("ngAfterViewInit called"); }
  ngOnDestroy(){ console.log("ngOnDestroy called"); }
}
```

Execution order:

Constructor → ngOnChanges → ngOnInit → ngDoCheck → ngAfterViewInit → ngOnDestroy

9 Responsibilities Quick Table

Layer	Responsibility
Angular Component	UI logic
Angular Service	API calls
HttpClient	Send HTTP request
Controller	Receive request
Service	Business logic
Repository	Database operations
Database	Store data

10 Quick Recall Version

Component → Service → HttpClient → Controller → Service → Repository → Database
Database → Repository → Service → Controller → Angular Service → Component → UI